



Programming Fundamentals Lab Manual



Experiment No. :11

Pointers in C + Dynamic Memory

Allocation - I

Objective:

The objectives of this lab are:

1. To understand pointer variables and the behavior of pointer operators (& and *).
2. To use pointers to pass arguments to functions by reference, enabling functions to modify actual variables.
3. To explore the close relationship among pointers, arrays, and strings, and how memory addresses are used in these structures.
4. To use pointers to functions and understand how function addresses can be stored and invoked dynamically.
5. To define, initialize, and manipulate arrays of strings using pointers to characters.

Prerequisites

Following are the prerequisites of this lab are:

- Understanding of variables, data types, and memory representation in C.
- Knowledge of how functions work and how arguments are passed by value.
- Basic understanding of arrays and strings, including indexing and storage.
- Using address-of (&) and dereference (*) operators in simple expressions.
- Familiarity with the concept of memory addresses and how they relate to variables.
- Understanding the difference between compile-time memory (stack) and dynamic memory (heap).

Introduction

A pointer is one of the most powerful and fundamental features of the C programming language. Pointers allow programs to directly access and manipulate memory by storing the addresses of variables rather than their actual values. This enables efficient data handling, dynamic memory allocation, and the creation of complex data structures.

Pointers make it possible to simulate pass-by-reference in functions, work with arrays and strings more efficiently, and even pass functions as arguments. They form the foundation for advanced concepts such as linked lists, stacks, queues, trees, and other dynamic data structures, which grow and shrink at runtime.

In this lab, we focus on understanding the basic concepts of pointer variables, pointer operators, memory representation, dereferencing, and their practical use in C programming. Mastering these



Programming Fundamentals Lab Manual



concepts is essential before moving toward advanced topics like structures, dynamic memory management, and pointer-based data structures.

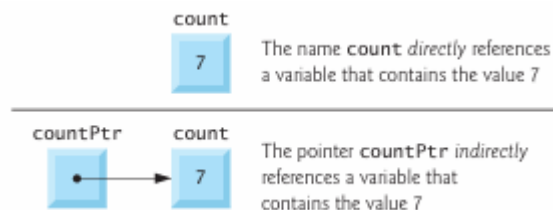
Declaring Pointers

Pointers, like all variables, must be defined before they can be used. The definition

```
int *countPtr, count;
```

Understanding Direct and Indirect Access to Variables

The upper part shows the integer variable `count` (type `int`) directly containing the value 7. The lower part shows the pointer variable `countPtr` (declared as `int *countPtr`) that holds the address of `count`, thus indirectly referencing the same value 7 through the pointer indirection operator. This illustrates that `countPtr` is a pointer to an integer, while `count` is an integer variable itself.



Understanding Pointer Initialization

Pointers should always be initialized at declaration to prevent undefined behavior. They can be set to NULL (preferred), 0, or the address of a valid variable using `&`. Initializing to NULL clearly indicates that the pointer currently points to nothing. Although 0 is automatically converted to the null pointer constant, using NULL is the recommended modern practice for better readability and safety. Leaving a pointer uninitialized results in a garbage value, which is a common cause of crashes.

Pointer Operators

The address-of operator `&` obtains the memory location of variable `y`. The statement `yPtr = &y;` stores this address in the pointer variable `yPtr`. After execution, `yPtr` contains the address 600000 and points to the integer variable `y` (value 5) located at that address. This allows `yPtr` to indirectly access and modify the value of `y` using the indirection operator `*`.

```
y = 5;
&yPtr;
yPtr = &y;
```



The Indirection (*) Operator

The unary `*` operator, commonly referred to as the indirection operator or dereferencing operator, returns the



Programming Fundamentals Lab Manual



value of the object to which its operand (i.e., a pointer) points.

For example, the statement

```
printf( "%d", *yPtr );
```

prints the value of variable y, namely 5. Using * in this manner is called dereferencing a pointer

Demonstrating the & and * Operators:

we use printf with the %p specifier to display memory addresses in hexadecimal format. The output shows that:

- &a (address of variable a) and aPtr (the pointer holding that address) print the exact same value – this confirms that aPtr = &a; successfully stored the address of a.
- The operators & and * are perfect inverses of each other:
 - *aPtr gives the value stored at the address held by aPtr (i.e., the value of a)
 - &*aPtr first dereferences, then takes address → returns aPtr itself
 - *&a first takes address, then dereferences → returns the original value of a

Thus, applying & and * consecutively in any order always yields the original pointer or value, demonstrating that these two operators cancel each other out.

This behavior is a fundamental property of pointers in C and proves that the pointer is correctly pointing to the target variable.

Example

Demonstrating & and * Pointer Operators

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a;                // a is an integer variable
```

```
    int *aPtr;            // aPtr is a pointer to an integer
```

```
    a = 7;                // assign value 7 to variable a
```

```
    aPtr = &a;            // aPtr gets the address of variable a
```

```
    printf("The address of a is %p\n", &a);
```

```
    printf("The value of aPtr is %p\n\n", aPtr);
```

```
    printf("The value of a is %d\n", a);
```

```
    printf("The value of *aPtr is %d\n\n", *aPtr);
```

```
    printf("Showing that * and & are complements of each other:\n");
```

```
    printf("&aPtr = %p\n", &*aPtr);    // same as aPtr
```

```
    printf("&&a = %p\n", *&a);        // same as a
```

```
    return 0;
```

```
}
```



Programming Fundamentals Lab Manual



Output

The address of a is 0x7fff5fbff83c
The value of aPtr is 0x7fff5fbff83c

The value of a is 7
The value of *aPtr is 7

Showing that * and & are complements of each other:
&*aPtr = 0x7fff5fbff83c
*&a = 7

Passing Arguments to Functions by Reference

In C programming, arguments can be passed to functions in two ways:

1. pass-by-value
2. pass-by-reference.

All arguments in C are inherently passed by value, meaning a copy of the argument's value is sent to the function. However, to simulate pass-by-reference (which allows modifying the original variable in the caller), pointers and the indirection operator (*) are used. This is useful for modifying caller variables or efficiently passing large data structures without copying.

Pass-by-reference, the address of the variable (obtained via the address-of operator &) is passed to the function. Inside the function, the pointer dereferences this address to access and modify the original value.

Pass-By-Value Example

This demonstrates passing an integer by value; the function cubes a copy, and the original remains unchanged.

```
Cube a variable using pass-by-value.
#include <stdio.h>
int cubeByValue(int n); // prototype

int main(void) {
    int number = 5; // initialize number
    printf("The original value of number is %d\n", number);
    number = cubeByValue(number); // pass number by value
    printf("The new value of number is %d\n", number);
    return 0;
}

// calculate and return cube of integer argument
int cubeByValue(int n) {
    return n * n * n; // cube local variable n and return result
}
```



Programming Fundamentals Lab Manual



Output:

```
The original value of number is 5
The new value of number is 125
```

Pass-By-Reference Example

Here, the address of the variable is passed, allowing the function to modify the original value directly.

Example:

```
Cube a variable using pass-by-reference with a pointer argument.
#include <stdio.h>

void cubeByReference(int *nPtr); // function prototype

int main(void) {
    int number = 5; // initialize number
    printf("The original value of number is %d\n", number);

    cubeByReference(&number); // pass address of number
    printf("The new value of number is %d\n", number);
    return 0;
}

// calculate cube of *nPtr; modifies number in main
void cubeByReference(int *nPtr) {
    *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
}
```

Output:

```
The original value of number is 5
The new value of number is 125
```

Using the const Qualifier with Pointers

Pointers can be combined with the const keyword in four different ways to control whether the pointer itself and/or the data it points to can be modified.

These four cases are explained below with examples:

1. **Non-constant pointer to non-constant data**
2. **Constant pointer to non-constant data**



Programming Fundamentals Lab Manual



3. Non-constant pointer to constant data
4. Constant pointer to constant data

Difference Between them :

Type	Can modify data?	Can modify pointer?	Typical Use Case
Non-const pointer to non-const data	Yes	Yes	General-purpose pointers
Const pointer to non-const data	Yes	No	Array parameters that must stay fixed
Non-const pointer to const data	No	Yes	Function parameters that should not change data (e.g., string processing)
Const pointer to const data	No	No	Read-only data, string literals

Example 1: Non-constant pointer to non-constant data

```
// Converting a string to uppercase using a
// non-constant pointer to non-constant data.
#include <stdio.h>
#include <ctype.h>

void convertToUppercase(char *sPtr); // prototype

int main(void) {
    char string[] = "cHaRaCters and $32.98"; // initialize char array

    printf("The string before conversion is: %s", string);
    convertToUppercase(string);
    printf("\nThe string after conversion is: %s\n", string);
} // end main
// convert string to uppercase letters
void convertToUppercase(char *sPtr)
```



Programming Fundamentals Lab Manual



```
{
```

The string before conversion is: cHaRaCters and \$32.98
The string after conversion is: CHARACTERS AND \$32.98

Output:

Example 2: Non-constant pointer to Constant data

```
#include <stdio.h>
void printCharacters(const char *sPtr);

int main(void) {
    char string[] = "print characters of a string"; // initialize char array

    puts("The string is:");
    printCharacters(string);
    puts("");
} // end main

// sPtr cannot modify the character to which it points,
// i.e., sPtr is a "read-only" pointer
void printCharacters(const char *sPtr) {
    for (; *sPtr != '\0'; ++sPtr) { // no initialization
        printf("%c ", *sPtr);
    } // end for
} // end function printCharacters
```

Output:

The string is:
p r i n t c h a r a c t e r s o f a s t r i n g

Example 3: Constant pointer to non-constant data

```
// Attempting to modify a constant pointer to non-constant data.
#include <stdio.h>
```

```
int main(void) {
    int x; // define x
```



Programming Fundamentals Lab Manual



Output:

```
c:\examples\ch07\fig07_13.c(15) : error C2166: l-value specifies const object
```

Example 4: Constant pointer to Constant data

```
// Attempting to modify a constant pointer to constant data.
#include <stdio.h>

int main(void) {
    int x = 5; // initialize x
    int y; // define y

    // ptr is a constant pointer to a constant integer. ptr always
    // points to the same location; the integer at that location
    // cannot be modified
    const int *const ptr = &x; // initialization is OK

    printf("%d\n", *ptr);
    *ptr = 7; // error: *ptr is const; cannot assign new value
    ptr = &y; // error: ptr is const; cannot assign new address
} // end main
```

Output:

```
c:\examples\ch07\fig07_14.c(16) : error C2166: l-value specifies const object
c:\examples\ch07\fig07_14.c(17) : error C2166: l-value specifies const object
```



Programming Fundamentals Lab Manual

Bubble Sort Using Pass-by-Reference

Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. In C, arrays are always passed to functions by reference (the address of the first element is passed). To actually swap two elements of the original array, the swap function must receive **pointers (addresses)** to those elements and use the indirection operator ***** this is true **pass-by-reference** behavior using pointers.

Example 5:

```
#include <stdio.h>

int main()
{
    int a = 50, b = 10, c = 30;
    int *p, *q;           // two pointers

    printf("Before sort: %d %d %d\n", a, b, c);

    // Simple sorting of 3 numbers using pointers
    if(a > b) { p = &a; q = &b; *p = *q; a = b; b = *p; }
    if(a > c) { p = &a; q = &c; *p = *q; a = c; c = *p; }
    if(b > c) { p = &b; q = &c; *p = *q; b = c; c = *p; }

    printf("After sort : %d %d %d\n", a, b, c);

    return 0;
}
```

Output:

```
Before sort: 50 10 30
After sort : 10 30 50
```

Size of Operator

The size, in bytes, of a variable or data type. It is extremely useful for writing portable code and for dynamic memory allocation. `sizeof(array)` gives the total size of the entire array, while `sizeof(pointer)` gives only the size of the pointer itself (usually 8 bytes on 64-bit systems).



Programming Fundamentals Lab Manual



Example 6:

```
#include <stdio.h>
#define SIZE 20

// This function receives a pointer but returns size of pointer only
size_t getSize(float *ptr)
{
    return sizeof(ptr);    // Warning: This gives size of pointer, not array!
}

int main()
{
    float array[SIZE];    // array of 20 floats

    printf("The number of bytes in the array is %u\n",
           sizeof(array)); // Correct → total size of whole array

    printf("The number of bytes returned by getSize is %u\n",
           getSize(array)); // Wrong → only size of pointer (usually 4 or 8)

    return 0;
}
```

Output:

```
The number of bytes in the array is 80
The number of bytes returned by getSize is 8
```



Programming Fundamentals Lab Manual



Lab Tasks

Task 1

Write a C program that:

1. Takes an array of 8 integers.
2. Prints the array before sorting.
3. Uses a function `swap(int *x, int *y)` that receives addresses and swaps the actual values using pointers.
4. Uses another function `bubbleSort()` that calls `swap(&arr[j], &arr[j+1])` whenever two adjacent elements are out of order.
5. Prints the array after sorting in ascending order.

Code

```
#include <stdio.h>

// swap using memory addresses
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// bubble sort algorithm
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            // if they're out of order swap them
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

int main() {
    // 8 random integers
    int arr[8] = {42, 12, 8, 99, 23, 5, 17, 1};
    int n = 8;

    printf("before sorting: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // after sorting it
    bubbleSort(arr, n);

    printf("after sorting: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```



Programming Fundamentals Lab Manual



Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ make task1
gcc -c task1.c
gcc -o task1 task1.o
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ ./task1
before sorting: 42 12 8 99 23 5 17 1
after sorting: 1 5 8 12 17 23 42 99
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ |
```

Task 2

Write a C program that uses the sizeof operator to display the number of bytes used by different data types and pointer types in memory. Your program should:

1. Declare variables of the following types:
char, short, int, long, long long, float, double, long double
2. Create an integer array of 20 elements and a pointer that points to this array.
3. Use printf and the sizeof operator to display the size (in bytes) of:
 - o Each variable type listed above
 - o The entire array
 - o The pointer variable
4. Run the program and observe how the sizes vary depending on system architecture (e.g., 32-bit or 64-bit).

Code:

```
1 #include <stdio.h>
2
3 int main() {
4     // basic types
5     char c;
6     short s;
7     int i;
8     long l;
9     long long ll;
10    float f;
11    double d;
12    long double ld;
13
14
15    int arr[20];
16    int *ptr = arr;
17
18    // using %zu because sizeof returns a size_t
19    printf("size of char: %zu bytes\n", sizeof(c));
20    printf("size of short: %zu bytes\n", sizeof(s));
21    printf("size of int: %zu bytes\n", sizeof(i));
22    printf("size of long: %zu bytes\n", sizeof(l));
23    printf("size of long long: %zu bytes\n", sizeof(ll));
24    printf("size of float: %zu bytes\n", sizeof(f));
25    printf("size of double: %zu bytes\n", sizeof(d));
26    printf("size of long double: %zu bytes\n", sizeof(ld));
27
28    printf("\n");
29    // comparing the whole array vs just the pointer
30    printf("size of the entire array (20 ints): %zu bytes\n", sizeof(arr));
31    printf("size of the pointer variable: %zu bytes\n", sizeof(ptr));
32
33    return 0;
34 }
```



Programming Fundamentals Lab Manual



Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ make task2
gcc -c task2.c
gcc -o task2 task2.o
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ ./task2
size of char: 1 bytes
size of short: 2 bytes
size of int: 4 bytes
size of long: 8 bytes
size of long long: 8 bytes
size of float: 4 bytes
size of double: 8 bytes
size of long double: 16 bytes

size of the entire array (20 ints): 80 bytes
size of the pointer variable: 8 bytes
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$
```

Task 3

Fix all the syntax and logical errors in the following code, and write the correct version of this C program with its output.

Bugged Code

```
#include <stdio.h>
#define SIZE 5

void wrongFunction(int *ptr)
{
    printf("Array size inside function = %d\n",
sizeof(ptr)/sizeof(ptr[0])); // Error 1
    *ptr[0] = 100;
    ptr = ptr + 3;
    *ptr = 999;
}

int main()
{
    int arr[SIZE] = {10, 20, 30, 40, 50};
    int *p = arr;

    printf("First value = %d\n", *arr[0]);

    wrongFunction(arr);

    return 0;
}
```



Programming Fundamentals Lab Manual



Corrected Code

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 void fixedFunction(int *ptr) {
5     // pointers only know their own size, so this just does pointer size / int size
6     // usually 8 / 4 = 2 on 64-bit systems. added the missing parenthesis here too.
7     printf("size calculation inside function (pointer/int) = %zu\n", sizeof(ptr) / sizeof(ptr[0]));
8
9     // changed *ptr[0] to ptr[0] since ptr[0] is already an int value, not an address
10    ptr[0] = 100;
11
12    // move pointer 3 spots forward
13    ptr = ptr + 3;
14    *ptr = 999;
15 }
16
17 int main() {
18     int arr[SIZE] = {10, 20, 30, 40, 50};
19
20     // arr[0] is just a raw value, took out the unnecessary * operator
21     printf("first value = %d\n", arr[0]);
22
23     // pass the array to the function
24     fixedFunction(arr);
25
26     // printing the array just to see what the function actually did
27     printf("\narray after function call:\n");
28     for(int i = 0; i < SIZE; i++) {
29         printf("%d ", arr[i]);
30     }
31     printf("\n");
32
33     return 0;
34 }
```

Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ make task3
gcc -c task3.c
gcc -o task3 task3.o
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$ ./task3
first value = 10
size calculation inside function (pointer/int) = 2

array after function call:
100 20 30 999 50
umar@umarr:~/Downloads/PF-Labs-main/Lab11_2025_EE_016$
```



Programming Fundamentals Lab Manual

